J. Concurrency & Coordination

Standalone capability brief · Suggested module: **Coordination SDK + Runtime**

Purpose

Provide clear, bounded coordination primitives for services and workers that share resources, while avoiding unrealistic guarantees such as universal exactly-once execution.

1. Scope and capability areas

Optimistic concurrency

Use versions, ETags, compare-and-swap, or conditional writes to prevent lost updates.

Leases and locks

Provide time-bounded distributed leases with fencing tokens where stale holders could cause damage.

Duplicate detection

Deduplicate messages and commands using stable idempotency keys and bounded retention.

Ordering and partitioning

Define per-key ordering guarantees and partition strategy for messages and events.

Conflict resolution

Document conflict detection and domain-owned resolution policies.

Backpressure

Use bounded queues, concurrency limits, and load shedding to protect dependencies.

Fairness

Support per-tenant and per-resource concurrency caps to prevent noisy-neighbor effects.

2. Shared contracts and standards

- Every coordination primitive states its consistency model, expiry behavior, and failure assumptions.
- Locks are not substitutes for idempotency or transactional storage constraints.
- Consumers explicitly document at-most-once, at-least-once, or other delivery semantics.
- Fencing tokens or equivalent safeguards are used when an expired lock holder could still act.

3. Recommended implementation approach

- Prefer database constraints and optimistic concurrency when sufficient.
- Use distributed locks only for operations that cannot be safely coordinated through simpler mechanisms.
- Apply idempotency keys at side-effect boundaries and retain deduplication records for a documented period.
- Configure bounded queues, consumer concurrency, retry limits, and tenant fairness.
- Test network partitions, worker pauses, lock expiry, duplicate delivery, and out-of-order messages.

4. Failure behavior and operational expectations

- If lock ownership cannot be confirmed, protected writes must reject stale fencing tokens.
- On queue saturation, apply documented backpressure or load shedding rather than unbounded memory growth.
- Duplicate messages should be harmless, observable, and safe to retry.

5. Suggested operational metrics

- Lock/lease contention and expiry
- Duplicate message rate
- Queue depth and age
- Optimistic conflict rate
- Per-tenant fairness and throttling events

6. Readiness and acceptance checklist

- ■ Concurrent updates cannot silently overwrite each other.
- ■ Duplicate delivery does not duplicate protected side effects.
- ■ Queue saturation has bounded and documented behavior.
- ■ Coordination guarantees are described accurately and tested under failure.

Ownership note: assign a named team, service owner, support route, versioning policy, and deprecation process before treating this capability as a platform contract.